17. RAG

22 March 2026 12:17

in focus → ability of large models to access and precisely manipulate knowledge

RAG models → combine pre-trained parametric and non parametric memory for language generation
- parametric → pre trained model
- non parametric → dense vector index accessed with a pre trained neural retriever

RAG is not just vector DB + retrieval + LLM
- not just a tool
- → it is part of the learning system

→ parametric models store knowledge in weights

precise access & updating knowledge is hard

not just retrieval difficulty
- staleness
- Lack of provenance
- imprecision in recall

Hard query → distⁿ over documents
- ↓ generation conditioned on documents

retrieval → a latent variable over documents
- Not just some Top k chunks
- But probabilistic selection of documents

Remove non parametric memory → Model suffers from Capacity limited and non-addressable recall

without external memory & if multiple facts compete for same parameters ?

↓ Shared parameter space

Interference between representations
- ↳ updating one fact can destroy another
- Rare facts get overwritten by frequent patterns

Compression bottleneck
- many facts → compressed into limited dimensions
- ↓ exact recall becomes approximate
- retrieval becomes pattern matching not lookup

↓ Bias towards high probability patterns
- loss of low frequency facts
- confidence without grounding

↳ Hallucination → surface level effect

↓ i. Capacity bound recall
- Knowledge is compressed into weights
- Retrieval becomes approximate, entangled, non indexable

ii. No selective access
- you cannot point to where knowledge lives
- retrieve specific facts on demand

iii. No explicit grounding
- leads to hallucination
- generation is not conditioned on verifiable external evidence

Recall failure → root cause

Hallucination → Observable effect

Lack of grounding → Mechanism gap

External Memory should be

1 Explicit
- facts exist as separate entries
- Not compressed into weights

2 Addressable
- You can retrieve specific pieces given a query
- Not rely on distributed recall

3 Non interfering
- one fact shouldn't overwrite other
- No gradient interference

4 Updatable
- Add / remove without retraining

Motivation for hybrid Memory
- combine parametric + non parametric

Benefits
- updatable
- inspectable
- expandable

Retrieval is not perfect
- it is learned + approximate
- performance depends on
  - → retriever quality
  - → representation quality

latent document means
- → the model doesn't observe which document is correct
- it must consider multiple documents possible

$d \sim p(d|x)$

The model optimizes

$$p(y|x) = \sum_d P(y|x,d)\, p(d|x)$$

The generator still learns from retrieved documents, even if they are latent

The model doesn't know which document is correct, so it learns by weighing multiple retrieved docs based on how well they support the output

Intuition
- Retriever proposes docs
- Generator evaluates usefulness via likelihood
- Training reinforces documents that help generate correct output

Gradient flow

if retriever is not differentiable
- ↳ No gradient flow from generation → retrieval
  - Generator learns $P(y|x,d)$
  - Retriever $P(d|x)$
  - retriever is trained independently ∴ is not differentiable
  - no signal of which docs actually helped generation
  - → Retriever might optimize similarity
    - But generator needs Useful for generation documents
  - → no credit assignment

use fail for generation documents

→ no credit assignment without differentiability
no way to assign credit/blame to retrieval

training retrieves documents that help generate correct output

Gradient flow
Even though documents are latent
if a document helps generate correct y
→ its probability increases

Implicit Supervision

Marginalization $p(y|x) = \sum_z P(y|x,z)\, p(z|x)$
you dont pick one doc
you average over all possible documents
Weighted by $p(z|x)$
i.e consider all docs but trust some more than others

RAG Seq → One doc explains whole answer
One latent variable z for entire sequence
Then marginalize $p(y|x)$

RAG Token → diff docs explain diff parts
Different z_i per token
More flexible
More expensive

for retrieval
$p(z|x) \propto \exp(q(x)^T d(z))$
Similarity = Inner product
higher similarity = higher probability
Basically softmax over document similarities

Why RAG sequence decoding is hard?
$P(y|x) = \sum_z P(y|x,z)\, p(z|x)$
Cannot decompose nicely per token
Beam search expects
$\prod_i P(y_i|\ )$
They run beam search per document
Combine scores across documents

In fast decoding
They ignore documents that didn't produce the sequence
↳ reduces compute

Gradient flows through generator - retriever
via log-sum-exp structure

HNSW here
hierarchical navigable small world graph
purpose efficient approximate nearest neighbor search

input seq 'x'
retrieve text documents 'z'
use them as additional context
when generating target sequence 'y'

To train generator & retriever end to end, we treat the retrieved document as a latent variable

(i) retriever $p_\eta(z|x)$
with parameters η that returns (top-k truncated) distributions over text passages given a query x

(ii) generator $p_\theta(y_i|x,z,y_{1:i-1})$
parametrized by θ that generates a current token based on context of previous tokens $y_{1:i-1}$, the original input x & a retrieved passage z

Two approaches
(i) the model uses the same document to predict each target token
(ii) can predict each target token based on a different document

RAG Seq model –
retrieved document as a single latent variable
top K docs retrieved
Generator produces output seq prob for each document which are then marginalized

$$P_{RAG\ seq}(y|x) \approx \sum_{z \in top\text{-}k(p(\cdot|x))} P_\eta(z|x)\, P_\theta(y|x,z)$$

$$= \sum_{z \in top\,k(p(\cdot|x))} P_\eta(z|x) \prod_i^N P_\theta(y_i|x,z,y_{1:i-1})$$

RAG Token model
we can draw a diff latent document for each target token & marginalize accordingly
Allows generator to choose from several docs when producing an answer

top k documents are retrieved
generator produces a dist^n for the next output token for each document before marginalizing and repeat

$$P_{RAG\ Token}(y|x) = \prod_i^N \sum_{z \in top\,k\,p(\cdot|x)} P_\eta(z|x)\, P_\theta(y_i|x,z,y_{1:i-1})$$

Retriever Dense passage Retriever
↓ follows a bi-encoder architecture

generator BART encoder-decoder seq-seq 400M

$p_\eta(z|x) \propto \exp(d(z)^T q(x))$
$d(z) = BERT_d(z)$ → dense representation of documents produced by document encoder
$q(x) = BERT_q(x)$ → query representation produced by query encoder

Maximum inner product search problem
Calculating top K $(p_\eta(\cdot|x))$ the list of k docs with highest prior probability

Calculating top K ($p_\eta(z|x)$) the list of k docs with highest prior probability

Training —

Minimize negative marginal log-likelihood of each target $\sum_j -\log P(y_j|x_j)$ using SGD with adam

Document encoder is fixed

Only fine-tuning query encoder $BERT_q$ & BART generator

Decoding

RAG seq, RAG Token require different ways to approximate $\arg\max p(y|x)$

RAG Token → Can be seen as standard autoregressive seq2seq generator with transition probability $p'_\theta(y_i|x, y_{1:i-1})$

↳ we can plug this into a standard beam decoder

RAG sequence

the likelihood $p(y|x)$ doesn't break into a per token likelihood

↓ beam search for each document z

Score each hypothesis using $p_\theta(y_i|x, z, y_{1:i-1})$

we get as set Y → run additional forward passes for which y doesn't appear in the beam,

Multiply generator probability with $p_\eta(z|x)$

then sum the probabilities across beams for marginals

↓

Thorough Decoding

For longer output seq |Y| can be come large → more no of forward passes

for an efficient decoding $p_\theta(y|x, z_i) = 0$ where y wasn't generated during beam search from x, z_i

↓

Thus avoids additional forward passes

↓

"Fast decoding"

Standard LM factorization

$$p(y|x) = \prod_t p(y_t | y_{<t}, x)$$

perfect for beam search

RAG Sequence

$$p(y|x) = \sum p(z|x)\, p(y|x, z)$$

you can't push sum inside product easily

probability of full sequence depends on

→ entire sequence

→ all documents

Trade offs

Aspect	Sequence	Token
modeling flexibility	low	high
Compute	low	high
Noise	low	high
Decoding complexity	high	low
Training complexity	moderate	higher

(worse overall practicality)

Decoding is expensive & messy

↳ RAG sequence (marginalization over docs at sequence level)

purpose efficient approximate nearest neighbor search

why? dataset = millions of docs

exact search → slow

Graph structure over embeddings

↳ allows fast traversal

↳ approximate top-K retrieval

RAG is not retrieval + generation pipeline

↓

It is latent variable generative model over documents

which enables

end to end training

Credit assignment

probabilistic retrieval

Retrieval doesn't find relevant docs

↳ it learns documents that maximize generation likelihood

if documents ↑ retrieval cost, marginalization cost, beam search exploration } Bottlenecks

Trade offs: more docs → better recall

harder optimization

Slower inference

Beam search refresh

approximate inference for sequence generation

goal $\arg\max_y p(y|x)$

exact search impossible

eg Vocabsize 50^k, seq len ~20 — Total seq 50000^{20} (intractable)

Greedy decoding (baseline)

At each step $y_t = \arg\max(p(y_t|y_{<t}, x))$

locally optimal ≠ globally optimal

Here instead of one sequence → keep K sequences

beam width = K

At time step t

1 you have k partial sequences

2 for each expand with all possible next tokens

3 Score all candidates

4 Keep top K sequences

→ entire sequence
→ all documents

They do →
i. Run beam search per document
ii. Get candidate sequences
iii. Rescore using full marginal

expensive because
each seq must be evaluated across documents
not just token by token

Decoding is expensive & messy
↳ RAG sequence
(marginalization over docs at sequence level)

RAG Token
↳ fits autoregressive factorization
can do standard beam search
no seq level recomputation
efficient decoding & natural

tokens
3 Score all candidates
& Keep top k sequences

Score is
$\log p(y_1 \ldots y_t) = \sum_i \log p(y_i | y_{<i})$

Greedy = single path
Beam = multiple competing hypotheses

Gradient flow through Marginalization

How gradients update
retriever $p_\eta(z|x)$
generator $p_\theta(y|x,z)$

Loss $\mathcal{L} = -\log \sum_z p_\theta(y|x,z) p_\eta(z|x)$

Gradient wrt retriever

$$\nabla_\eta \mathcal{L} \propto \sum_z \underbrace{p(z|x,y)}_{\text{posterior}} \nabla_\eta \log p_\eta(z|x)$$

Retriever is trained using posterior over documents
↳ Documents that help generate correct output get reinforced

Posterior v Prior (Retrieval distribution)

$p(z|x)$
but training $p(z|x,y)$

even if retriever initially assigns low prob
→ if doc explains well, posterior boosts it

Reality we approximate $\sum_z \rightarrow \sum_{z \in \text{top}\,k}$

Consequence → biased gradients
limited exploration
missing good documents
} model can only learn from retrieved documents

Retriever optimizes → embedding similarity
Generator needs → useful for generation documents
↳ Semantic similarity ≠ usefulness for generation

* Production RAG

pipeline (query → embedding → retrieve → append → LLM)
a non differentiable pipeline

Paper RAG
$p(y|x)$ is a joint probabilistic model

Paper	Production
Soft combination over documents	hard top k selection
Probabilistic learned retriever	fixed retriever
	no joint training

} hard approximation of the latent variable model

Paper $\sum p(y|x,z) p(z|x)$
production
$z^* = \text{top}\,k(x)$
$y \sim p(y|x,z^*)$
→ no marginalization
no posterior learning
gradient to retriever

final line

Production RAG pipelines approximate a latent variable generative model by replacing probabilistic marginalization with top-k retrieval and independent components